

ESCOLA_API

Documentacao Tecnica do Backend

Projeto Escola Conectada: API REST em ASP.NET Core para autenticao JWT, autorizacao por perfis, CRUD escolar, caderneta digital, calendario escolar, agenda de avaliacoes e trabalhos com notificacoes aos alunos, QR Code bancario ficticio, holerites de funcionarios, uploads, persistencia com Entity Framework Core, Swagger, Scalar, logs e testes.

Escopo: todo o backend localizado em `ESCOLA_API`, incluindo arquitetura, camadas, classes, regras de negocio, diagramas e tecnologias.

Formato: documentacao tecnica em PDF gerada a partir deste HTML.

Projeto: **ESCOLA_API**

Repositorio: **SPA-Vue / ESCOLA_API**

Data: **28/05/2026**

Documento: **ESCOLA_API/docs/documentacao-tecnica-backend.pdf**

Indice

1. Visao geral do backend
2. Tecnologias usadas e versoes
3. Tipo de arquitetura
4. Funcao de cada camada
5. Comunicacao entre camadas
6. Diagramas de fluxo, classes e entidade-relacional
7. Funcao de cada classe por camada
8. Regras de negocio
9. Autenticacao, autorizacao e JWT
10. Migrations, FluentValidation, Swagger, Scalar, logs e testes
11. CRUDs e endpoints

1. Visao Geral do Backend

O backend `ESCOLA_API` e uma API REST para uma aplicacao escolar. Ele centraliza o cadastro de usuarios, caderneta digital, disciplinas, calendario escolar, eventos de avaliacao/trabalho, QR Code bancario ficticio, holerites de funcionarios, notificacoes e arquivos de perfil, alem de cuidar da autenticao e da autorizacao por perfis.

Dominio

Gestao escolar com usuarios, perfis, disciplinas, caderneta, calendario, eventos, holerites, notificacoes e arquivos.

Interface externa

Controllers HTTP documentados via Swagger/OpenAPI e Scalar.

Persistencia

EF Core com SQL Server/Azure SQL e SQLite para desenvolvimento/testes.

Seguranca

JWT Bearer, roles, hash de senha e troca obrigatoria da senha padrao.

Qualidade

FluentValidation, testes unitarios e teste contra SQL bruto.

Observabilidade

Logs diarios, notificacoes persistidas e consumidor opcional do Azure Service Bus.

2. Tecnologias Usadas e Versoes

Tecnologia	Versao	Uso no backend
.NET / ASP.NET Core	<code>net10.0</code>	Framework principal da API Web.
Microsoft.AspNetCore.Authentication.JwtBearer	<code>10.0.8</code>	Validacao de tokens JWT no pipeline.
Entity Framework Core	<code>10.0.8</code>	ORM, DbContext, migrations e consultas LINQ.
EF Core SQL Server	<code>10.0.8</code>	Provider do banco principal.
EF Core SQLite	<code>10.0.8</code>	Provider usado no ambiente de desenvolvimento e em testes.
FluentValidation.AspNetCore	<code>11.3.1</code>	Validacao automatica de ViewModels.
Swashbuckle.AspNetCore	<code>10.1.7</code>	Swagger/OpenAPI e Swagger UI.
Scalar.AspNetCore	<code>2.14.14</code>	Documentacao interativa alternativa em <code>/scalar</code> .
QRCode	<code>1.8.0</code>	Geracao de QR Code PNG para dados bancarios ficticios do aluno.
Azure.Messaging.ServiceBus	<code>7.20.1</code>	Consumo opcional de eventos de notificacao.
Azure.Storage.Blobs	<code>12.28.0</code>	Storage de fotos de perfil, certificados PDF e holerites PDF.
SQL Server	<code>2022-latest</code>	Banco em Docker Compose.
Docker .NET SDK/Runtime	<code>10.0</code>	Build e runtime da imagem da API.
xUnit	<code>2.9.3</code>	Framework de testes unitarios.
Moq	<code>4.20.72</code>	Mocks nos testes de controllers/services.
Microsoft.NET.Test.Sdk	<code>17.14.1</code>	Execucao dos testes .NET.
coverlet.collector	<code>6.0.4</code>	Coleta de cobertura quando habilitada.

3. Tipo de Arquitetura

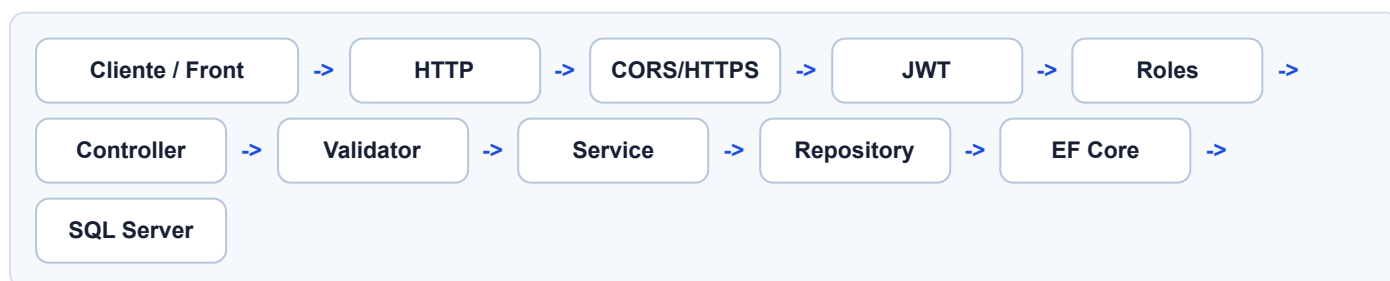
A arquitetura é uma arquitetura em camadas. Essa escolha separa responsabilidades e deixa o código mais simples de testar: a camada de API não sabe detalhes do banco, os services concentram regras, o DataContext configura o modelo e os ViewModels isolam contratos HTTP das entidades persistidas.

API	Aplicacao	Infraestrutura	Contratos/Dominio
Controllers, middlewares, JWT, autorizacao, Swagger, Scalar e codigos HTTP.	Services e interfaces que executam regras e orquestram dados.	DataContext, Repository, migrations, logging, hashing e acesso ao banco.	Models, ViewModels, validators e mapeamentos.

4. Funcao de Cada Camada

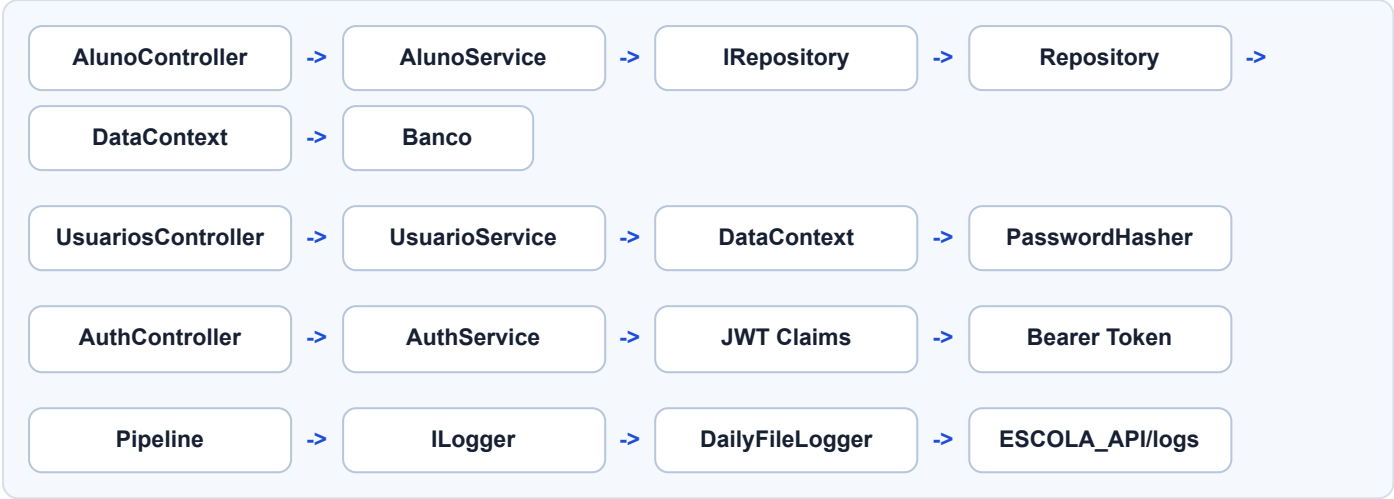
Camada	Pastas	Funcao
Inicializacao	Program.cs	Registra DI, controllers, validators, JWT, policies, EF Core, Swagger, Scalar, logs, CORS e migrations no startup.
API	Controllers	Recebe HTTP, chama services, aplica atributos de autorizacao e retorna status codes.
Aplicacao	Services	Executa regras de negocio, consulta dados, cria/atualiza/exclui entidades e retorna ViewModels.
Dados	Data	Configura EF Core, relacionamentos, seeds, repository e consultas.
Dominio	Models	Representa entidades persistidas no banco.
Contratos	ViewModels	Define payloads de entrada e saida expostos pela API.
Validacao	Validators	Valida ViewModels antes da execucao das regras de aplicacao.
Seguranca	Security	Hash, verificacao de senha e regra de senha padrao.
Documentacao	Swagger e Scalar	Expor OpenAPI, Swagger UI e Scalar API Reference para endpoints protegidos.
Logs	Logging	Provider de arquivo diario para o sistema de logging do ASP.NET Core.

5. Diagrama de Fluxo da Requisicao



O retorno percorre o caminho inverso: banco, EF Core, repository, service, ViewModel, controller e resposta HTTP. O middleware de logs registra inicio, fim, status e tempo de execucao de cada requisicao.

6. Diagrama de Comunicacao Entre Camadas



7. Fluxo de Autenticacao JWT



O token carrega `sub`, `email`, `nameidentifier`, `name`, `id_perfil` e `role`. A role vem da descricao do perfil e alimenta os atributos `[Authorize(Roles = "...")]`.

8. Diagrama de Classes



CadernetaDigital • IdCadernetaDigital • IdAlunoUsuario • IdDisciplina • Notas • Presencas • Faltas	UsuarioArquivo • IdArquivo • IdUsuario? • NomeBlob • TipoArquivo • NomeOriginal • Url • ContentType • TamanhoBytes	Notificacao • IdNotificacao • IdUsuario • Tipo/Titulo/Mensagem • Link • MediaAritmetica • Situacao • Lida
Holerite • IdHolerite • IdUsuario • CompetenciaMes • CompetenciaAno • NomeOriginal • NomeBlob • Url	Professor • Id • Nome • IdUsuario? • Usuario • Alunos	Aluno • Id • Nome • Sobrenome • DataNasc • ProfessorId • IdUsuario?
Diretoria • Id • Nome • IdUsuario? • Usuario	DisciplinaEvento • IdEventoDisciplina • IdDisciplina • Tipo • Titulo • Descricao • Data	Services • AuthService • UsuarioService • CadernetaDigitalService • DisciplinaEventoService • CalendarioEscolarService • AlunoQrCodeBancarioService • HoleriteService • NotificacaoService • UsuarioArquivoService

Relacionamentos principais: Perfil 1:N Usuario , Usuario 1:N Disciplina , Usuario 1:N CadernetaDigital , Usuario 1:N UsuarioArquivo , Usuario 1:N Holerite , Usuario 1:N Notificacao , Disciplina 1:N CadernetaDigital e Disciplina 1:N DisciplinaEvento .

9. Modelo Entidade-Relacional

PERFIL

- IdPerfil PK
- DescricaoPerfil

USUARIO

- IdUsuario PK
- Nome
- Email UK
- Telefone
- DataNascimento
- NomeMae
- NomePai
- Endereco
- Senha
- FotoPerfilUrl
- IdPerfil FK

DISCIPLINA

- IdDisciplina PK
- Nome
- IdProfessorUsuario FK

CADERNETADIGITAL

- IdCadernetaDigital PK
- IdAlunoUsuario FK
- IdDisciplina FK
- Notas
- Presencas
- Faltas

ARQUIVO

- IdArquivo PK
- IdUsuario FK opcional
- NomeBlob
- TipoArquivo
- Url
- ContentType

NOTIFICACAO

- IdNotificacao PK
- IdUsuario FK
- Titulo
- Mensagem
- Link
- Lida

HOLERITE

- IdHolerite PK
- IdUsuario FK
- CompetenciaMes
- CompetenciaAno
- NomeOriginal
- NomeBlob
- Url
- ContentType

PROFESSOR

- Id PK
- Nome
- IdUsuario FK opcional

ALUNO

- Id PK
- Nome
- Sobrenome
- DataNasc
- ProfessorId FK obrigatoria
- IdUsuario FK opcional

DIRETORIA

- Id PK
- Nome
- IdUsuario FK opcional

DISCIPLINAEVENTO

- IdEventoDisciplina PK
- IdDisciplina FK
- Tipo
- Titulo
- Descricao
- Data
- CriadoEmUtc

Relacao	Cardinalidade	Delete behavior
Perfil -> Usuario	1:N	Restrict
Professor -> Aluno	1:N	Cascade
Usuario -> Aluno	1:N opcional	SetNull
Usuario -> Professor	1:N opcional	SetNull
Usuario -> Diretoria	1:N opcional	SetNull
Usuario -> Disciplina	1:N obrigatoria	Restrict
Disciplina -> CadernetaDigital	1:N obrigatoria	Cascade
Disciplina -> DisciplinaEvento	1:N obrigatoria	Cascade
Usuario -> CadernetaDigital	1:N obrigatoria	Restrict
Usuario -> Arquivo	1:N opcional	SetNull
Usuario -> Holerite	1:N obrigatoria	Cascade
Usuario -> Notificacao	1:N obrigatoria	Cascade

10. Função de Cada Classe por Camada

10.1 Inicialização

Classe/arquivo	Camada	Função
<code>Program.cs</code>	Bootstrap/API	Configura DI, controllers, JSON, FluentValidation, CORS, JWT, policies, Swagger, Scalar, EF Core, migrations automáticas, logs, uploads e middleware de requisições.

10.2 Controllers

Classe	Função	Principais ações
<code>AlunoController</code>	Endpoint HTTP de alunos.	Listar, buscar por id, buscar por professor, criar, atualizar e excluir aluno.
<code>ProfessorController</code>	Endpoint HTTP de professores.	Listar, buscar por id, criar, atualizar e excluir professor.
<code>DiretoriaController</code>	Endpoint HTTP da diretoria escolar.	Listar, buscar por id, criar, atualizar e excluir integrante da diretoria.
<code>UsuariosController</code>	Endpoint HTTP de usuários, perfis e arquivos.	Listar usuários, buscar por id, listar perfis, criar, atualizar, excluir, enviar foto, listar/baixar certificados e excluir arquivos.
<code>CadernetaDigitalController</code>	Endpoint HTTP da caderneta digital.	Listar, buscar, criar, atualizar e excluir lançamentos; manter disciplinas e eventos de avaliação/trabalho do professor.
<code>CalendarioEscolarController</code>	Endpoint HTTP do calendário escolar.	Retorna ano completo, mês selecionado e feriados nacionais brasileiros.
<code>AlunoQrCodeBancarioController</code>	Endpoint HTTP do QR Code bancário fictício.	Gera dados bancários fictícios, QR Code PNG em base64 e links de compartilhamento para o aluno logado.
<code>HoleritesController</code>	Endpoint HTTP de holerites.	Lista e baixa holerites do funcionário logado; administradores enviam, listam, baixam e removem PDFs de funcionários.
<code>NotificacoesController</code>	Endpoint HTTP de notificações.	Listar minhas notificações, contar não lidas, criar notificação manual e marcar como lida.
<code>AuthController</code>	Endpoint HTTP de autenticação.	Login, usuário atual, validação simples de token, validação admin e alteração de senha.

10.3 Services e Interfaces

Classe/interface	Função	Observação
<code>IALunoService</code>	Contrato de operações de aluno.	Permite injeção e testes da camada controller.
<code>AlunoService</code>	Regra de aplicação para aluno.	Usa repository, converte ViewModel para Model e retorna AlunoViewModel.
<code>IProfessorService</code>	Contrato de professor.	Abstrai listagem, busca, criação, edição e exclusão.
<code>ProfessorService</code>	Regra de aplicação para professor.	Orquestra repository e preserva vínculo opcional com usuário.
<code>IDiretoriaService</code>	Contrato de diretoria.	Expõe CRUD da administração escolar.
<code>DiretoriaService</code>	Regra de aplicação para diretoria.	Cria, atualiza e remove integrantes da diretoria.
<code>IUsuarioService</code>	Contrato de usuário.	Define CRUD e listagem de perfis.
<code>UsuarioService</code>	Regra de aplicação para usuários.	Valida email único, perfil existente, define senha padrão, aplica permissões por perfil e gera notificações de cadastro/atualização.
<code>ICadernetaDigitalService</code>	Contrato da caderneta digital.	Define operações de disciplinas e lançamentos com base no usuário logado.
<code>CadernetaDigitalService</code>	Regra da caderneta digital.	Aplica regras de professor/aluno, impede associação duplicada, calcula média/situação e gera notificações.
<code>IDisciplinaEventoService</code>	Contrato de eventos de disciplina.	Define listagem e CRUD de avaliações e trabalhos escolares por disciplina.
<code>DisciplinaEventoService</code>	Regra de eventos de disciplina.	Garante que professor altere apenas eventos das próprias disciplinas, que aluno veja apenas disciplinas associadas e que alunos matriculados sejam notificados em criação/atualização.
<code>ICalendarioEscolarService</code>	Contrato de calendário escolar.	Define geração do calendário anual com mês selecionado e feriados nacionais.
<code>CalendarioEscolarService</code>	Regra de calendário escolar.	Calcula meses, dias, finais de semana, Páscoa de Cristo e feriados nacionais fixos.
<code>IALunoQrCodeBancarioService</code>	Contrato de QR Code bancário fictício.	Define geração dos dados fictícios e links de compartilhamento.
<code>AlunoQrCodeBancarioService</code>	Regra de QR Code bancário fictício.	Gera dados determinísticos por aluno, QR Code PNG, Data URL, mailto e link do WhatsApp.

<code>IHoleriteService</code>	Contrato de holerites.	Define listagem, upload, download e remocao de holerites de funcionarios.
<code>HoleriteService</code>	Regra de holerites.	Permite que professor/administrador consultem os proprios holerites, que administradores gerenciem PDFs de funcionarios e que todos os professores sejam notificados quando um holerite for lancado pelo administrador.
<code>INotificacaoService</code>	Contrato de notificacoes.	Lista, conta e marca notificacoes como lidas.
<code>NotificacaoService</code>	Regra de notificacoes.	Controla notificacoes por usuario e permite envio manual individual ou por perfis pelo administrador.
<code>IUsuarioArquivoService</code>	Contrato de arquivos de usuario.	Define upload/download/listagem/remocao de foto e certificados.
<code>UsuarioArquivoService</code>	Regra de arquivos de usuario.	Valida permissao, tipo/tamanho de arquivo e persiste metadados no banco.
<code>ServiceBusNotificacaoWorker</code>	Worker opcional.	Consome mensagens do Azure Service Bus e grava notificacoes no banco quando configurado.
<code>IAuthService</code>	Contrato de autenticao.	Login, usuario atual e troca de senha.
<code>AuthService</code>	Regra de autenticao e JWT.	Verifica senha, gera token, monta claims e altera senha do usuario autenticado.

10.4 Data e Persistencia

Classe/interface	Funcao	Detalhes
<code>DataContext</code>	DbContext do EF Core.	Declara DbSet, tabelas, chaves, indices, FKs, delete behavior e seeds.
<code>IRepository</code>	Contrato de persistencia generica e consultas.	Abstrai Add, Update, Delete, SaveChanges e consultas de aluno/professor/diretoria.
<code>Repository</code>	Implementacao de acesso a dados.	Usa LINQ, Include, AsNoTracking, ordenacao e filtros por id/professor.
<code>LocalUsuarioArquivoStorage</code>	Storage local.	Salva arquivos em disco durante desenvolvimento.
<code>AzureBlobUsuarioArquivoStorage</code>	Storage Azure.	Salva fotos, certificados e holerites PDF no Azure Blob Storage em producao.

10.5 Models

Classe	Funcao	Campos relevantes
Aluno	Entidade persistida de aluno.	Nome, Sobrenome, DataNasc, ProfessorId, IdUsuario.
Professor	Entidade persistida de professor.	Nome, IdUsuario, lista de Alunos.
Diretoria	Entidade persistida de pessoa da administracao escolar.	Nome e IdUsuario opcional.
Usuario	Entidade autenticavel.	Nome, Email, Telefone, DataNascimento, NomeMae, NomePai, Endereco, Senha hash, FotoPerfilUrl, IdPerfil e vinculos de arquivos/notificacoes/caderneta/holerites.
Perfil	Entidade de autorizacao.	Administrador, Professor e Aluno.
Disciplina	Entidade de disciplina da caderneta.	Nome, professor responsavel, cadernetas e eventos.
DisciplinaEvento	Entidade de evento escolar.	Disciplina, tipo, titulo, descricao, data, criacao e atualizacao.
CadernetaDigital	Entidade de notas e frequencia.	Aluno, disciplina, notas serializadas, presencas e faltas.
UsuarioArquivo	Entidade de metadados de arquivo.	NomeBlob, TipoArquivo, NomeOriginal, Url, ContentType, tamanho e data.
Holerite	Entidade de holerite de funcionario.	Usuario, competencia, NomeBlob, NomeOriginal, Url, ContentType, tamanho e data de criacao.
Notificacao	Entidade de notificacao.	Titulo, mensagem, link, situacao, media, flags de leitura e origem.

10.6 ViewModels e Mapeamentos

Classe	Funcao
<code>AlunoCreateEditViewModel</code>	Payload de criacao/edicao de aluno.
<code>AlunoViewModel</code>	Resposta completa de aluno com professor e usuario.
<code>AlunoSummaryViewModel</code>	Resumo de aluno usado em professor.
<code>ProfessorCreateEditViewModel</code>	Payload de criacao/edicao de professor.
<code>ProfessorViewModel</code>	Resposta de professor com usuario e alunos.
<code>ProfessorSummaryViewModel</code>	Resumo de professor usado em aluno.
<code>DiretoriaCreateEditViewModel</code>	Payload de criacao/edicao da diretoria.
<code>DiretoriaViewModel</code>	Resposta da diretoria com usuario vinculado.
<code>UsuarioCreateViewModel</code>	Payload de criacao/edicao de usuario sem expor senha, com data de nascimento, nome da mae, nome do pai e endereco.
<code>UsuarioSummaryViewModel</code>	Dados publicos de usuario sem senha, incluindo dados cadastrais e responsaveis.
<code>UsuarioArquivoViewModel</code>	Resposta dos metadados de arquivos enviados pelo usuario.
<code>CadernetaDigitalCreateUpdateViewModel</code>	Payload de lancamento de notas, presencas e faltas.
<code>CadernetaDigitalViewModel</code>	Resposta da caderneta com aluno, email, disciplina, notas, media aritmetica, situacao e cor.
<code>DisciplinaCreateUpdateViewModel</code>	Payload de cadastro/edicao de disciplina.
<code>DisciplinaViewModel</code>	Resposta de disciplina com professor responsavel.
<code>DisciplinaEventoCreateUpdateViewModel</code>	Payload de cadastro/edicao de avaliacao ou trabalho escolar.
<code>DisciplinaEventoViewModel</code>	Resposta de evento escolar vinculado a disciplina e professor.
<code>CalendarioEscolarAnoViewModel</code>	Resposta do calendario anual com meses, dias e feriados nacionais.
<code>AlunoQrCodeBancarioViewModel</code>	Resposta com dados bancarios ficticios, QR Code e links de compartilhamento.
<code>HoleriteViewModel</code>	Resposta com metadados do holerite, competencia, funcionario, URL e tamanho do PDF.
<code>NotificacaoCreateViewModel</code>	Payload de notificacao manual.
<code>NotificacaoPerfisCreateViewModel</code>	Payload de notificacao manual em lote para perfis selecionados ou todos os perfis.
<code>NotificacaoEnvioViewModel</code>	Resposta do envio em lote com total e notificacoes criadas.
<code>NotificacaoViewModel</code>	Resposta de notificacao para o painel do usuario.
<code>PerfilViewModel</code>	Contrato de saida para perfis.
<code>LoginRequestViewModel</code>	Payload de login.
<code>AuthResponseViewModel</code>	Resposta do login com token, expiracao, usuario e flag de senha padrao.
<code>AlterarSenhaViewModel</code>	Payload para troca de senha.
<code>MappingExtensions</code>	Metodos de conversao entre Models e ViewModels.

10.7 Validators

Classe	Funcao	Regras principais
<code>AlunoCreateEditViewModelValidator</code>	Valida aluno.	Nome, sobrenome, data <code>dd/MM/yyyy</code> , professor positivo e usuario positivo quando informado.
<code>ProfessorCreateEditViewModelValidator</code>	Valida professor.	Nome obrigatorio e usuario positivo quando informado.
<code>DiretoriaCreateEditViewModelValidator</code>	Valida diretoria.	Nome obrigatorio e usuario positivo quando informado.
<code>UsuarioCreateViewModelValidator</code>	Valida usuario.	Nome, email valido, telefone, data de nascimento, responsaveis/endereco opcionais e perfil positivo.
<code>UsuarioUpdateViewModelValidator</code>	Valida edicao de usuario.	Nome, email, telefone, data de nascimento, responsaveis/endereco opcionais e perfil conforme permissao do usuario logado.
<code>DisciplinaCreateUpdateViewModelValidator</code>	Valida disciplina.	Nome obrigatorio com limite de tamanho.
<code>DisciplinaEventoCreateUpdateViewModelValidator</code>	Valida evento de disciplina.	Tipo Avaliacao/Trabalho, titulo, descricao e data obrigatoria.
<code>CadernetaDigitalCreateUpdateViewModelValidator</code>	Valida lancamento da caderneta.	Aluno, disciplina, notas, presencas e faltas com limites validos.
<code>LoginRequestViewModelValidator</code>	Valida login.	Email valido e senha obrigatoria.
<code>AlterarSenhaViewModelValidator</code>	Valida troca de senha.	Senha atual, nova senha forte, confirmacao e bloqueio da senha padrao.

10.8 Security, Swagger, Scalar e Logging

Classe	Camada	Funcao
<code>PasswordHasher</code>	Security	Gera hash PBKDF2-SHA256 com salt e verifica senha informada.
<code>DefaultPasswordPolicy</code>	Security	Define a senha padrao e detecta se o hash ainda corresponde a ela.
<code>AuthorizeOperationFilter</code>	Swagger	Marca endpoints protegidos no OpenAPI e adiciona respostas 401/403.
<code>MapScalarApiReference</code>	Scalar	Publica a documentacao interativa em <code>/scalar</code> usando o mesmo OpenAPI do Swagger.
<code>FileLoggerOptions</code>	Logging	Configura diretorio, prefixo e nivel minimo de log.
<code>FileLoggerExtensions</code>	Logging	Expõe <code>AddDailyFileLogger</code> para registrar o provider no builder.
<code>DailyFileLogger</code>	Logging	Implementa <code>ILogger</code> e envia mensagens para o provider.
<code>DailyFileLoggerProvider</code>	Logging	Cria arquivo diario e grava linhas de log com data, nivel, categoria, evento, escopo e execucao.

11. Regras de Negocio

- Todo endpoint de negocio exige token JWT valido.
- **Administrador** cadastra, edita e exclui usuarios; gerencia holerites de funcionarios; visualiza caderneta digital, disciplinas, eventos e calendario escolar.
- **Professor** consulta alunos e professores, edita apenas o proprio perfil, consulta seus proprios holerites, cadastra disciplinas, lanca notas/frequencia e agenda avaliacoes/trabalhos.
- **Aluno** consulta e edita apenas o proprio perfil, visualiza somente cadernetas/eventos associados ao seu usuario e gera QR Code bancario ficticio proprio.
- Somente administrador cria e exclui usuarios.
- Somente administrador lista perfis para cadastro de usuario.
- Quando um administrador cadastra usuario, o usuario criado recebe notificacao com os dados cadastrados.
- Quando um usuario atualiza seus dados, o administrador criador recebe notificacao com os dados corrigidos.
- Usuarios criados pela API recebem a senha inicial `Senha@252525`.
- Cadastro de usuario aceita `DataNascimento`, `NomeMae`, `NomePai` e `Endereco` como dados cadastrais opcionais.
- Quando o login detecta uso da senha padrao, a resposta retorna `deveAlterarSenhaPadrao = true`.
- Senha nunca trafega em respostas e nunca e persistida em texto puro.
- Email de usuario e unico; duplicidade gera erro de regra de negocio.
- Perfil informado no cadastro/edicao de usuario precisa existir.
- Disciplina nao pode ser duplicada para o mesmo professor, mesmo com diferenca de maiusculas/minusculas.
- Professor pode criar, editar e excluir eventos apenas das proprias disciplinas; os tipos aceitos sao `Avaliacao` e `Trabalho`.
- Ao criar ou atualizar avaliacao/trabalho, a API cria notificacoes para os alunos matriculados na disciplina pela caderneta digital.
- Eventos de disciplina usam datas ISO `yyyy-MM-dd`, adequadas para Datpicker com digitacao ou selecao visual.
- Aluno pode ser associado a varias disciplinas, desde que a associacao aluno/disciplina seja unica.
- Aluno visualiza eventos somente das disciplinas associadas a ele pela caderneta digital.
- Calendario escolar retorna ano completo, mes selecionado e feriados nacionais brasileiros.
- QR Code bancario do aluno usa dados ficticios e links de compartilhamento; nao executa pagamento real nem envio direto por provedor externo.
- QR Code bancario e exclusivo de aluno; professor e administrador usam o modulo de holerites quando houver PDFs vinculados.
- Holerites aceitam apenas PDF, sao limitados a 10 MB, possuem competencia mes/ano e podem ser vinculados somente a professores e administradores; o lancamento por administrador gera notificacoes para todos os professores com competencia, arquivo, funcionario e responsavel pelo lancamento.
- A caderneta calcula media aritmetica das notas e retorna situacao/cor: reprovado, recuperacao, aprovado ou reprovado por faltas quando atingir 10 faltas.
- Fotos, certificados e holerites sao salvos em storage local ou Azure Blob; no banco ficam apenas URL e metadados do arquivo.
- Notificacoes ficam persistidas na tabela `Notificacao`; o Service Bus e opcional para consumo de mensagens externas.
- Administrador pode enviar notificacao manual para um usuario especifico ou em lote para perfis como `Aluno`, `Professor` e `Administrador`.
- Aluno precisa apontar para professor valido.
- `IdUsuario` em aluno, professor e diretoria e opcional; quando informado deve ser positivo.
- Data de nascimento de usuario e opcional, nao pode ser futura e deve trafegar no JSON como `yyyy-MM-dd`.
- Ao excluir usuario, vinculos opcionais em aluno, professor e diretoria ficam nulos.
- Ao excluir professor, os alunos vinculados seguem cascade conforme configurado no relacionamento obrigatorio.

12. CRUDs e Endpoints

Entidade	Metodo e rota	Autorizacao	Funcao
Auth	POST <code>/api/Auth/login</code>	Publico	Autentica e retorna JWT.
Auth	GET <code>/api/Auth/me</code>	Autenticado	Retorna usuario atual.
Auth	POST <code>/api/Auth/alterar-senha</code>	Autenticado	Altera senha.
Auth	POST <code>/api/Auth/esqueci-senha</code>	Publico	Fluxo de recuperacao de senha.
Usuarios	GET/POST/PUT/DELETE <code>/api/usuarios</code>	Admin gerencia; professor consulta alunos/professores e edita proprio perfil; aluno edita proprio perfil.	CRUD centralizado de usuarios.
Perfis	GET <code>/api/usuarios/perfis</code>	Administrador	Lista perfis disponiveis.
Arquivos	GET/POST/DELETE <code>/api/usuarios/{id}/foto</code> e <code>/api/usuarios/{id}/arquivos</code>	Proprio usuario, administrador e leituras permitidas por perfil.	Upload/download/listagem de foto e certificados.
QR Code bancario	GET <code>/api/alunos/me/qr-code-bancario</code>	Aluno	Gera dados bancarios ficticios, QR Code PNG e links de compartilhamento do aluno logado.
Holerites	GET <code>/api/holerites/me</code> e GET <code>/api/holerites/me/{holeriteId}/download</code>	Administrador e Professor	Lista e baixa os holerites do funcionario logado.
Holerites	GET/POST/DELETE <code>/api/holerites/usuarios/{usuarioId}</code> e download por <code>{holeriteId}</code>	Administrador	Gerencia holerites PDF de professores e administradores.
Calendario escolar	GET <code>/api/calendario-escolar?ano=2026&mesSelecioneado=5</code>	Autenticado	Retorna calendario anual com mes destacado e feriados nacionais brasileiros.
Caderneta	GET <code>/api/caderneta-digital</code>	Admin visualiza tudo; professor visualiza seus lancamentos; aluno apenas os seus.	Lista lancamentos conforme perfil.
Caderneta	POST/PUT/DELETE <code>/api/caderneta-digital</code>	Professor	Cria, altera e remove lancamentos de notas/frequencia.
Disciplinas	GET/POST/PUT/DELETE <code>/api/caderneta-digital/disciplinas</code>	Professor escreve; admin/aluno consultam conforme permissao.	Mantem disciplinas vinculadas aos professores.
Eventos de disciplinas	GET/POST/PUT/DELETE <code>/api/caderneta-digital/disciplinas/eventos</code> e <code>/api/caderneta-digital/disciplinas/{disciplinaId}/eventos</code>	Professor escreve; admin/aluno consultam conforme permissao.	Agenda avaliacoes e entregas de trabalhos por disciplina e notifica alunos matriculados.
Notificacoes	GET/PATCH <code>/api/notificacoes</code>	Autenticado	Lista, conta e marca notificacoes do usuario logado.

Notificacoes	POST <code>/api/notificacoes</code>	Administrador	Envia notificacao manual para um usuario.
Notificacoes	POST <code>/api/notificacoes/perfis</code>	Administrador	Envia notificacao manual em lote para usuarios dos perfis informados.

13. Migrations e Banco

Migration	Funcao
<code>20260521160031_InitialSqlServer</code>	Cria estrutura inicial com Professores e Alunos.
<code>20260521194537_SeedFiftyRecords</code>	Insere massa inicial maior de professores e alunos.
<code>20260521202457_AddJwtAuthUsuariosDiretoria</code>	Adiciona JWT no modelo: Perfil, Usuario, Diretoria, FKs IdUsuario, indices, seeds de 3 perfis e 20 usuarios.
<code>20260525125946_AddCadernetaDigital</code>	Adiciona disciplinas, lancamentos de caderneta, notas e frequencia.
<code>20260526204546_AddNotificacoesArquivosUsuario</code>	Adiciona notificacoes, arquivos de usuario e foto de perfil.
<code>20260526212447_AjustaTabelaArquivos</code>	Ajusta a tabela <code>Arquivo</code> para metadados flexiveis.
<code>20260527182246_AddUsuarioCriadorNotificacoes</code>	Registra o administrador criador do usuario para notificacoes de atualizacao.
<code>20260528180602_AddUsuarioDataNascimentoDisciplinaEventos</code>	Adiciona data de nascimento em usuario e tabela de eventos de disciplina.
<code>20260528190707_AddHoleritesFuncionarios</code>	Adiciona tabela de holerites de funcionarios com competencia, metadados do PDF e vinculo ao usuario.
<code>20260528210505_AddCalendarioEscolarEventos</code>	Adiciona eventos escolares institucionais ao calendario.
<code>20260528222740_AddUsuarioResponsaveisEndereco</code>	Adiciona nome da mae, nome do pai e endereco ao cadastro de usuario.

No startup, a API aplica migrations automaticamente quando usa SQL Server. Para SQLite, a API usa `EnsureCreated`.

14. Logging

Logs ficam em `ESCOLA_API/logs/escola-api-YYYYMMDD.log`. No Docker Compose, o volume `../ESCOLA_API/logs:/app/logs` grava os logs do container na pasta do projeto.

Origem	Evento
Startup	Ambiente, diretorio de logs e preparacao do banco.
Middleware	Inicio/fim de requisicao com metodo, rota, usuario, status e tempo.
Controllers	Excecoes completas em falhas de CRUD.
Auth	Login recusado, login aceito e troca de senha.

15. Swagger/OpenAPI e Scalar

Swagger e Scalar documentam rotas, payloads, status codes e segurança Bearer usando o mesmo documento OpenAPI. O filtro `AuthorizeOperationFilter` identifica endpoints protegidos e anexa o esquema JWT. Os comentarios XML dos controllers e ViewModels descrevem os contratos novos de calendario, eventos de disciplina, data de nascimento, QR Code bancario ficticio, holerites de funcionarios e notificacoes geradas por agenda.

Interface	Rota
Swagger UI	<code>/swagger</code>
OpenAPI JSON	<code>/swagger/v1/swagger.json</code>
Scalar API Reference	<code>/scalar</code>

1. Executar `POST /api/Auth/login`.
2. Copiar o token.
3. Clicar em `Authorize`.
4. Informar `Bearer {token}`.

16. Testes Unitarios

Os testes ficam em `ESCOLA_API/ESCOLA_API.Tests`.

Grupo	O que valida
Controllers	Respostas principais de Aluno e Usuarios.
Services	Criacao, hash de senha, duplicidade de email, login, troca de senha, QR Code, calendario, eventos de disciplina, notificacoes de agenda e holerites.
Validators	Regras de entrada de aluno, professor, usuario, eventos de disciplina e senha.
Security	Protecao contra uso de SQL bruto.

Comando: `dotnet test ESCOLA_API.Tests/ESCOLA_API.Tests.csproj`.

17. Execucao e Deploy Local

Acao	Comando
Rodar API local	<code>dotnet run</code>
Build	<code>dotnet build ESCOLA_API.csproj</code>
Testes	<code>dotnet test ESCOLA_API.Tests/ESCOLA_API.Tests.csproj</code>
Docker Compose	<code>docker compose up -d --build</code>
Swagger no Compose	<code>http://localhost:5001/swagger</code>
Scalar no Compose	<code>http://localhost:5001/scalar</code>

Observacao tecnica: a chave JWT e a senha do SQL Server nao ficam versionadas. Em desenvolvimento, preencha o arquivo local `.env` a partir de `.env.example`. Em ambiente real, use variaveis de ambiente ou secrets do provedor de deploy.